

Jobsheet Praktikum: Pertemuan 3

Enkapsulasi dan Konstruktor

Mata Kuliah	Praktikum Pemrograman Berbasis Objek (RTI253008)
Pertemuan	3 (Minggu 3)
Durasi	1 × 4 × 50' praktikum; 1 × 1 × 50' tugas/laporan mandiri

A. Capaian Praktikum

Setelah menyelesaikan jobsheet ini, mahasiswa mampu:

1. Menjelaskan risiko atribut publik, lalu menerapkan encapsulation dan information hiding (atribut private, akses lewat method).
2. Menulis getter dan setter, termasuk method yang memvalidasi nilai masukan.
3. Menulis constructor yang mewajibkan data lengkap dan menetapkan atribut read-only (getter tanpa setter).

B. Persiapan dan Prasyarat

- **Alat:** JDK 17 atau lebih baru, NetBeans (editor yang digunakan sepanjang praktikum ini).
- **Proyek:** pertemuan ini melanjutkan proyek bank-mini dari Pertemuan 2.
- **Verifikasi cepat** sebelum memulai:

```
java -version
javac -version
```

Apabila keduanya menampilkan nomor versi tanpa galat, proses dapat dilanjutkan.

Tanpa NetBeans? Jobsheet ini tetap dapat diikuti menggunakan editor teks biasa:

```
javac -d out src/id/ac/polinema/*.java
java -cp out id.ac.polinema.Main
```

Checkpoint dan output yang dihasilkan tetap sama persis, apa pun editor yang digunakan.

C. Langkah Kerja

Langkah 1: Melanjutkan Proyek bank-mini

Buka kembali proyek bank-mini dari Pertemuan 2. Jalankan proyek untuk memastikan kondisinya masih sesuai kondisi akhir Pertemuan 2 sebelum Account diubah pada langkah berikutnya.

- ✓ **Checkpoint:** program masih mengompilasi dan berjalan, menampilkan tiga baris - balance: seperti pada akhir Pertemuan 2 (Nadia, Budi, Sari).

Langkah 2: Menerapkan Encapsulation ke Account

Account dari Pertemuan 2 masih memiliki atribut publik ownerName dan balance. Kode lain bisa langsung menulis `acc.balance = -999999;`, tanpa melalui `deposit()/withdraw()`. Invariant paling dasar sebuah rekening, saldo yang selalu tervalidasi, jadi tidak terjamin sama sekali. Ganti isi `Account.java` sesuai diagram kelas yang telah dibahas di slide konsep: seluruh atribut dibuat private (information hiding lewat access modifier paling ketat yang masih memungkinkan kelas bekerja), tambahan atribut `accountNumber`, satu constructor yang mewajibkan data lengkap (nomor rekening, nama pemilik, saldo awal), getter untuk setiap atribut, serta `deposit()/withdraw()` yang memvalidasi nilai masukan dan mengembalikan boolean:

Karena constructor kini mewajibkan data lengkap, perbarui pengujian di `Main.java`:

- ✓ **Checkpoint:** program berhasil dikompilasi ulang dan menampilkan `A001 - Nadia - balance: 350000.0`.

```

1 package id.ac.polinema;
2
3 public class Account {
4     private String accountNumber;
5     private String ownerName;
6     private double balance;
7
8     public Account(String accountNumber, String ownerName, double balance) {
9         this.accountNumber = accountNumber;
10        this.ownerName = ownerName;
11        this.balance = balance;
12    }
13
14    public String getAccountNumber() {
15        return accountNumber;
16    }
17
18    public String getOwnerName() {
19        return ownerName;
20    }
21
22    public double getBalance() {
23        return balance;
24    }
25
26    public boolean deposit(double amount) {
27        if (amount <= 0) {
28            return false;
29        }
30        balance += amount;
31        return true;
32    }
33
34    public boolean withdraw(double amount) {
35        if (amount <= 0 || amount > balance) {
36            return false;
37        }
38        balance -= amount;
39        return true;
40    }
41
42    public void printInfo() {
43        System.out.println(accountNumber + " - " + ownerName + " - balance: " + balance);
44    }
45 }

```

Figure 1: Account.java setelah encapsulation diterapkan

```

1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Account acc = new Account("A001", "Nadia", 500000);
6         acc.withdraw(150000);
7         acc.printInfo();
8     }
9 }

```

Figure 2: Main.java memakai constructor Account yang baru

⚠ **Jika gagal:** apabila muncul galat constructor Account in class Account cannot be applied to given types, periksa apakah jumlah dan urutan argumen pada new Account(...) sudah sesuai dengan constructor yang tersedia.

Catatan. accountNumber sengaja hanya diberi getter, tanpa setter, mengikuti pola atribut read-only dari slide konsep. Nilainya ditetapkan sekali lewat constructor dan tidak pernah berubah lagi.

D. Tugas dan Deliverable

Kumpulkan hal berikut sesuai format yang diminta Dosen:

- Screenshot output program setelah Langkah 2.
- **Tugas mandiri:**
 1. Account yang kamu buat belum membatasi jumlah penarikan per hari. Tambahkan atribut privat dailyWithdrawalLimit (diisi lewat constructor), lalu ubah withdraw() agar juga menolak penarikan yang melebihi batas ini, selain aturan saldo yang sudah ada:

```
1 package id.ac.polinema;
2
3 public class Account {
4     private String accountNumber;
5     private String ownerName;
6     private double balance;
7     private double dailyWithdrawalLimit;
8
9     public Account(String accountNumber, String ownerName, double balance, double dailyWithdrawalLimit) {
10         this.accountNumber = accountNumber;
11         this.ownerName = ownerName;
12         this.balance = balance;
13         this.dailyWithdrawalLimit = dailyWithdrawalLimit;
14     }
15
16     public String getAccountNumber() {
17         return accountNumber;
18     }
19
20     public String getOwnerName() {
21         return ownerName;
22     }
23
24     public double getBalance() {
25         return balance;
26     }
27
28     public boolean deposit(double amount) {
29         if (amount <= 0) {
30             return false;
31         }
32         balance += amount;
33         return true;
34     }
35
36     public boolean withdraw(double amount) {
37         if (amount <= 0 || amount > balance || amount > dailyWithdrawalLimit) {
38             return false;
39         }
40         balance -= amount;
41         return true;
42     }
43
44     public void printInfo() {
45         System.out.println(accountNumber + " - " + ownerName + " - balance: " + balance);
46     }
47 }
```

Figure 3: Account.java dengan tambahan dailyWithdrawalLimit

Buktikan dengan membuat satu Account bersaldo 1000000 dan batas harian 200000 di Main, lalu coba tarik 300000 (harus ditolak) dan 150000 (harus berhasil):

```
1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Account limited = new Account("A005", "Fajar", 1000000, 200000);
6         System.out.println("Withdraw 300000 allowed? " + limited.withdraw(300000));
7         System.out.println("Withdraw 150000 allowed? " + limited.withdraw(150000));
8         limited.printInfo();
9     }
10 }
```

Figure 4: Main.java menguji dailyWithdrawalLimit

2. Jawab secara singkat (2-3 kalimat untuk masing-masing pertanyaan): (a) mengapa mengembalikan nilai boolean dari deposit()/withdraw() lebih aman dibandingkan tidak memberi tahu pemanggil sama sekali ketika nilainya ditolak? (b) sebutkan satu atribut pada Account yang menurutmu sebaiknya dibuat read-only (getter tanpa setter), dan jelaskan alasannya.

E. Kriteria Penilaian

Komponen	Bobot	Kriteria Lengkap (100%)	Kriteria Minimum
Langkah kerja tuntas	40%	Seluruh langkah dijalankan dan berfungsi	Sebagian besar langkah selesai, hasil akhir berjalan
Checkpoint terverifikasi	35%	Semua checkpoint tercapai dan dibuktikan (screenshot/output)	Sebagian checkpoint terbukti
Tugas mandiri	25%	Batas penarikan harian benar dan jawaban konsep tepat	Batas penarikan harian ada meski jawaban belum lengkap